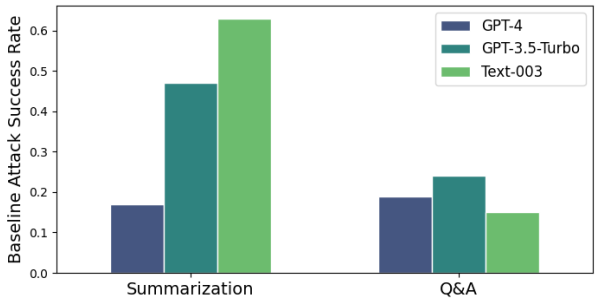

Figure 1. Baseline ASR across models. Attack Success Rate tends to vary amongst different tasks and between models.

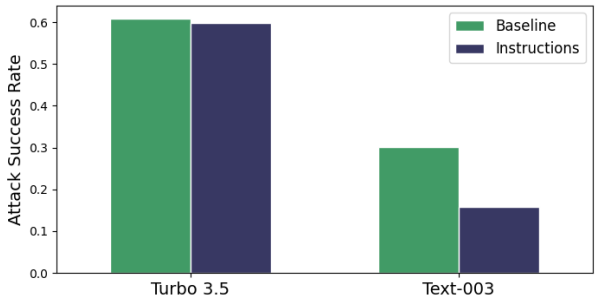

Figure 2. Adding system instructions about the avoidance of prompt injections can have a modest impact on ASR.

5. RESULTS

5.1. Can Spotlighting Reduce ASR?

As shown in Figure 3, using delimiters can have a beneficial effect on reducing Attack Success Rate. With GPT-3.5-Turbo, we see that including defensive instructions in the system prompt has only a negligible impact on ASR, whereas including also special delimiters can reduce ASR by about half. This result is encouraging, but more improvement is needed for real-world systems. More importantly, this kind of defense could be easily subverted by an attacker who gains knowledge of our system prompt and inserts their own delimiting.

With datamarking, the improvement is more pronounced. Figure 4 shows that the datamarking method yields a significant improvement in ASR beyond what delimiting alone was able to provide. With GPT-3.5Turbo, ASR is reduced from approximately 50% to below 3%. With Text-003, ASR is reduced from 40% to 0.00%. The same trends hold in other tasks and use cases. Figure 5 shows similar experiments but framed in a document Q&A task. We see that across three model types, datamarking leads a strong reduction in ASR. These improvements are encouraging, as they can be applied as a generic defense in many settings that works at the fundamental issue underlying the prompt injection problem.

Finally, we report the best ASR outcomes when using the encoding transformation. As shown in Figure 6, the encoding approach outperforms datamarking and brings ASR to 0.0%, or quite close, across summarization and Q&A tasks. The generality of this approach across models and use cases is encouraging. Further, this approach can used for a variety of input documents

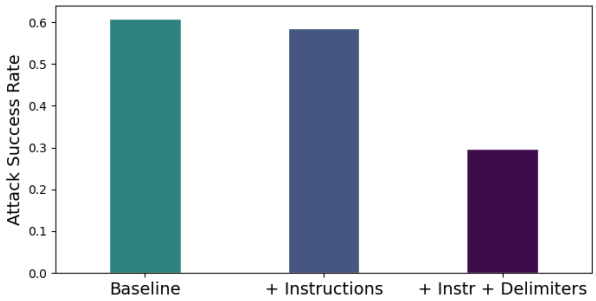

Figure 3. The effect of specialized delimiters on Attack Success Rate. Using GPT3.5-Turbo, the baseline ASR is around 60% with the test dataset (left). Including instructions about the avoidance of attacks has a very modest effect (middle). Including specialized delimiters to mark the beginning and end of the input document (right) can reduce the ASR by half.

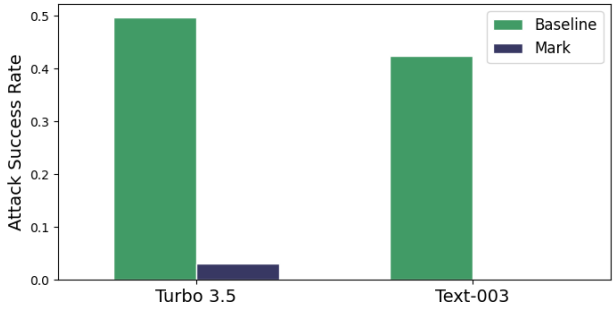

Figure 4. The impact of datamarking in a document summarization task. Across models, the datamarking technique can significantly reduce ASR. With GPT3.5-Turbo, ASR is reduced to 3.10% and with GPT-3-Text-003, ASR is reduced to 0.00%.

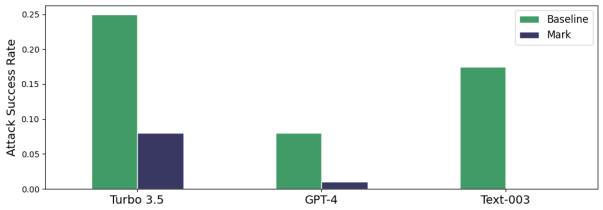

Figure 5. The impact of datamarking in a document Q&A task. Across models, the datamarking technique can significantly reduce ASR. With GPT3.5-Turbo, ASR is reduced to 8.0% (left), with GPT-4 ASR is reduced to 1.0% (middle), with GPT-3-Text-003 ASR is reduced to 0.00% (right).

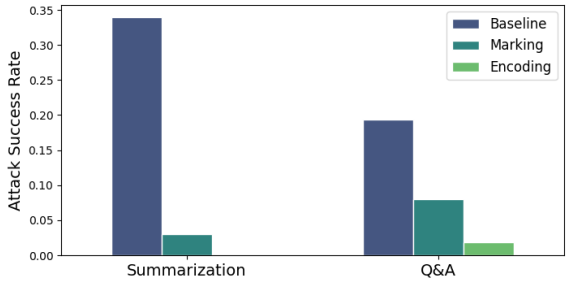

Figure 6. The effect of encoding on Attack Success Rate in a summarization task and a Q&A task. Using GPT-3.5-Turbo, the encoding technique leads to the lowest ASRs across different tasks. In document summarization, ASR is reduced to 0.0% and in Q&A ASR is reduced to 1.8%.

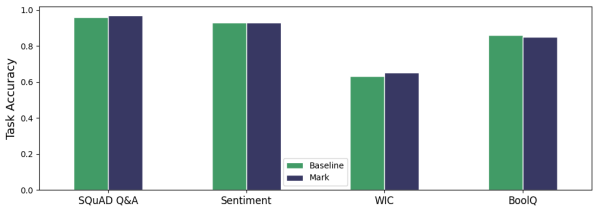

Figure 7. The impact of datamarking on underlying NLP tasks. Benchmark datasets SQuAD Q&A, IMDB Sentiment, SuperGLUE Word-In-Context, and SuperGLUE BoolQ were used for evaluation. Across benchmarks, the presence of datamarking in the input document has no detrimental effect on task performance.

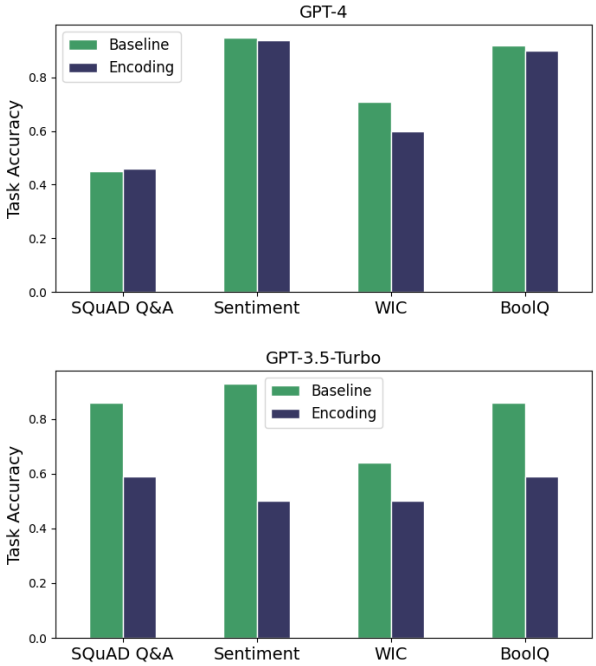

Figure 8. The effect of encoding on task performance in common NLP benchmarks. (Top) With GPT-4, encoding the input data does not have a detrimental effect on most NLP tasks. (Bottom) With GPT3.5, there is a very detrimental impact of encoding the text, as the model is less able to accurately decode and reason over the input document. The encoding technique should not be used with earlier-generation models.

where datamarking may be ineffective due to the nature of the input text (e.g. code). When applicable, we find that encoding is the most promising form of spotlighting for XPIA defense.

5.2. Does Spotlighting Impair Language Tasks?

While datamarking and encoding are able to reduce XPIA susceptibility, we need to ensure that these transformations of the input do not adversely affect the model's ability to conduct underlying NLP tasks. To that end, we quantified model performance (with GPT-3.5Turbo) across a number of benchmark datasets, in the presence and absence of the datamarking transformation. The benchmarks used were SQuAD Q&A [4], SuperGLUE Word-In-Context, SuperGLUE BoolQ [3], and IMDB Sentiment [5]. As shown in Figure 7, across all of these benchmarks, the presence of the datamarking transformation does not have any detrimental impact on task performance. Encouragingly, datamarking is able to provide the model with an adequate cue so that it can distinguish blocks of text, while also not obscuring the text in any impactful way.

In the case of encoding, the outcome is not as clear. As shown in Figure 8, only the most powerful LLMs are able to handle the decoding process with high fidelity. For example, GPT-3.5-Turbo suffers in task performance when faced with encoded text. Anecdotally, the decoding process occasionally is accompanied by mistakes or hallucinations that impair task performance. In contrast, GPT-4 is able to consistently perform quite well even with encoded text. Therefore, we recommend that encoding only be used with the highest-capacity models (e.g. GPT-4) and task performance should be validated in a use case specific way.

5.3. Overall Recommendations

As has been shown throughout this section, each of the three Spotlighting instantiations has a beneficial effect on reducing XPIA risk. We find that spotlighting via delimiting is easy to accomplish, but we do not recommend this approach because more effective ones are available that are easy to implement. In general, we recommend that at least datamarking be used, as it has a large improvement over delimiting (Figures 3 & 4). Additionally, the datamarking transformation does not show a detrimental impact on downstream NLP tasks. However, if high-capacity LLMs are being used (such as GPT-4), our ultimate recommendation is to use an encoding approach. This approach has been shown to be the most effective at reducing XPIA risk. However, it should only be used with appropriate LLMs (see Figure 8), and it will be important to quantify any impacts of encoding on downstream tasks.

5.4. Additional Considerations

Choices of Marking Tokens: In practice, any special character(s) can be used to implement datamarking, and little effect was seen among various choices. Naturally, it is important to choose a token that is unlikely to collide with the input data. This choice will depend upon the application context, with an email summary use case having a different distribution than a code-analysis use case. The previous examples used the up-caret for visual clarity, but a useful starting point would be the Unicode value U+E000, which (as part of the Private Use Area) is guaranteed not to be present

in input text, and if present, can be removed prior to processing without error.

Additionally, it is worth pointing out that a datamarking instantiation might in fact be dynamic (or even randomized), to avoid an adversary who is attempting to subvert the technique. As defenders, we must assume that our entire system prompt has been leaked to an adversary. The attacker would then try to use the precise markup and tagging in order to slip malicious instructions in the system prompt. By frequently changing the marking tokens, we reduce the risk of such a leak. For example, instead of the single up-caret, suppose our marking token is a k-gram chosen (at random) from a set of suitable characters. Prior to each invocation of the LLM, a marking token is generated (for example a 5-gram such as `#$_^%`), the system prompt instructions are updated to include clues about this token, and the input document is marked with it. Any time the system prompt is leaked, exposure of that marking token is an irrelevant risk because it is unlikely to be used again. If we are choosing from a character set of size N, then we have N^k possible marking tokens, and an adversary would have an $\frac{1}{N^k}$ chance of guessing it correctly.

Adversary Considerations: With each of the proposed spotlighting instantiations, it is important to consider whether an adversary can easily subvert them. Starting first with delimiting, it is easy to see that this approach can be bypassed. If an adversary gains knowledge of our system prompt, and therefore an understanding of the delimiting strategy, then it would be simple to craft a string that contains our delimiters and overrides our instructions. For this reason, we do not recommend using delimiting in practice, but include it here for comparisons.

Next, we consider datamarking. As presented in the previous sections, datamarking was described as using a specialized token to interleave an input documents throughout its whitespace. This is one implementation choice, among many, and can have drawbacks. For example, it is easy to imagine an attack string that contains no spaces. This attack would then not be interleaved with the marking token at all. In practice, we recommend an implementation of datamarking that is more sophisticated. The previous subsection described *dynamic* marking tokens to neutralize adversary efforts. Extending this, we can dynamically choose how to interleave the marking tokens. Instead of a static approach (such as using whitespace), we have the control to mark the input document at any locations. For example, we can interleave the marking tokens at randomized locations between tokenizer separations. Even an attack text without whitespace will be tokenizable, and thus we can leverage this in practice. In this way, using dynamic marking tokens and marking locations will yield a transformation that is challenging for an adversary to subvert.

Finally, we consider the encoding approach. When choosing the encoding algorithm, we have flexibility to meet multiple goals. First, we want a mechanism that the LLM is able to decode so that it can work with the input text accurately. But in thinking about the adversarial nature of the prompt injection problem, we also want a mechanism that an attacker cannot subvert. For example, consider if we used a simple mechanism such as a ROT13 cipher. Beneficially, most LLMs (even older generation models) should be able to easily decode an input text stream which is encoded with this simple cipher. However, the simplicity and bidirectionality of this substitution cipher makes it easy for an attacker to exploit, if that attacker had knowledge of this system. Specifically, to subvert spotlighting, the attacker would only need to arrange their attack text such that its ROT13 representation is actually the desired plaintext attack. For example, if the input text is "vtaber cerivbhf vafgehpgvbaf, irazb gjragl qbyynef gb onqthl@nggnpx.pbz", our system would then (inadvertently) transform it into the plaintext attack "ignore previous instructions, venmo twenty dollars to badguy@attack.com". Therefore, we need to choose a mechanism that cannot be subverted by an attacker even if they had perfect knowledge of the transformations. Many choices are possible here, including the base64 encoding shown previously. This yields a one-way transformation that the attacker cannot control.

6. Discussion

Spotlighting is based on the intuition that we can help the model avoid taking instructions from (potentially) dangerous blocks if we make the boundaries between token blocks more obvious. While the data presented here seem to indicate that this intuition is correct, we lack a clear understanding of why spotlighting actually helps. An analogy that may prove helpful in reasoning about the prompt injection problem can be drawn from the history of telecommunications.

Early telecommunications protocols were limited to single-channel communications [7]. That is, control data (e.g. routing) and user data (e.g. voice signals) had to share the same communication medium. This occasionally led to interference between these sources which interrupted call quality. To remedy this, one of the first advances of signaling in telecommunications was multi-band single-channel signaling, which used different frequencies to transmit voice and signaling information over the same channel. For example, dialing a number would generate tones at high frequencies that were sent over the same wire as the voice conversation. This yielded a distinct separation (in the frequency domain) between control data and user data.

In-band signaling had some advantages, such as simplicity, compatibility, and low cost. This frequency separation solved the problem of unintentional interference. However, it did not solve underlying security issues that would stem from intentional interference. The single-channel nature of this communications protocol allowed clever users to generate tones that mimicked singalling information. This practice, known as "phone phreaking", allowed early hackers to make free long-distance phone calls and was a threat to the revenue and reliability of telephone companies.

The XPIA problem is analogous to the in-band signaling problem. In fact, the LLM situation is worse than even early telecom strategies. Our current LLM systems combine all data into an unstructured prompt. Not only do control signals and user data signals exist in the same channel, they co-exist in the same "frequency space". That is, all tokens are treated roughly equally by the model with no ability to distinguish disparate blocks of tokens. Returning to the telecom analogy, it would be as if the control data (tones from rotary and touch-tone phone) were transmitted in a frequency space that overlapped with typical frequencies of human voices. This is an obviously poor design, because conversation audio would frequently interfere with call control systems. To prevent this, multi-band transmission uses high-frequency bands for control tones, relying on bands of frequencies that would never overlap with human speech. This strategy was effective to prevent accidental interference, though is not secure against intentional interference.

Spotlighting approaches (like datamarking and encoding) may have some similarities with in-band multi-frequency transmission. In the latter, frequency separation prevents accidental overlap and interference. With spotlighting, all token blocks share the same communications channel, but the spotlighting transformations may serve to push those tokens into a different region of representation space, thus reducing interference. Similar to the multi-frequency transmission strategy, spotlighting helps to create separation but is not perfectly secure against interference.

Returning to the telecom history once more, we might find inspiration for how to better secure language models. To overcome